UNIVERSIDADE FEDERAL DO CEARÁ

Test Smells em projetos OpenSource - Diário de Bordo

Disciplina: Qualidade de Software (2026.2 - T01).
Equipe: Felipe Maia (514914) e Guilherme Sousa (565684).
Projeto: pipeshub-ai/pipeshub-ai.
Fork: fel0psmya/pipeshub-ai.

SMELL 01	
Colaborador:	Felipe Maia
Localização:	• Arquivo: backend/python/tests/unit/modules/parsers/test_excel_parser.py • Método: test_mm_ss_with_time_context • Linha: 2650 • Commit: refactor(tests): remove Non-Functional Statement in test_mm_ss_with_t… · fel0psmya/pipeshub-ai@c3e73ec
Eu estou atualmente trabalhando na refatoração do seguinte test smell:	Non-Functional Statement. O método de teste contém docstring, comentários e a instrução pass, que não executa nenhuma operação funcional. Segundo o CATSPY, isso gera falso positivo nos resultados do teste e dificulta a manutenção. O smell está nas categorias Manutenibilidade e Confiabilidade.
Minhas principais dificuldades na remoção do test smell são:	O CATSPY prevê duas refatorações: adicionar a asserção que o método deveria ter ou removê-lo. Adicionar a asserção não é possível aqui, porque o teste pretendia cobrir um ramo de _resolve_ambiguous_format que é inalcançável (detalhado no SMELL 02). Restou a remoção do método. Outra dificuldade foi que o mesmo método também tinha um Unknown Test (linha 2635), e a remoção resolveu os dois smells em um único commit.
Usei o CATSPY para:	Confirmar a definição do smell, seus efeitos (falso positivo e dificuldade de manutenção) e as duas refatorações previstas, para escolher entre elas.
Página do CATSPY consultada:	CATSPY - Non-Functional Statement
Eu estou usando os seguintes métodos de refatoração para remover o test smell:	Remoção do método de teste sem finalidade, uma das duas opções indicadas pelo CATSPY para Non-Functional Statement.

UNIVERSIDADE FEDERAL DO CEARÁ

SMELL 02	
Colaborador:	Felipe Maia
Localização:	• Arquivo: backend/python/tests/unit/modules/parsers/test_excel_parser.py • Método: test_mm_ss_with_time_context • Linha: 2635 • Commit (mesmo do SMELL 01): refactor(tests): remove Non-Functional Statement in test_mm_ss_with_t… · fel0psmya/pipeshub-ai@c3e73ec
Eu estou atualmente trabalhando na refatoração do seguinte test smell:	Unknown Test. O método não tem nenhuma asserção, então só falharia se lançasse exceção, e como não executa nada, passa sempre. No CATSPY, o smell está nas categorias Manutenibilidade e Confiabilidade.
Minhas principais dificuldades na remoção do test smell são:	A refatoração indicada pelo CATSPY para Unknown Test é introduzir asserções explícitas, mas isso não era possível. O teste foi escrito para cobrir o ramo “mm:ss sem horas” de _resolve_ambiguous_format, e esse ramo é inalcançável: has_time só é verdadeiro se a entrada tem um h, e todo h vira %H/%I (ou continua h) antes da checagem de [hH%], que portanto sempre encontra correspondência. Sem um cenário que exercite o ramo, não há resultado a verificar, e o código de produção não pode ser alterado. A saída foi a remoção do método, a mesma do SMELL 01.
Usei o CATSPY para:	Confirmar a definição de Unknown Test (teste sem asserções claras, que passa se nenhuma exceção for lançada) e a refatoração prevista, para avaliar se era aplicável.
Página do CATSPY consultada:	CATSPY - Unknown Test
Eu estou usando os seguintes métodos de refatoração para remover o test smell:	Remoção do método de teste, por não ter asserção nem código alcançável a verificar. Essa é a refatoração indicada pelo CATSPY para Non-Functional Statement, aplicada aqui porque a prevista para Unknown Test (adicionar asserção) não era viável. Alternativa descartada: trocar o corpo por um teste de comportamento alcançável, que seria um teste novo

	e provavelmente redundante com test_single_m_in_time_context.

SMELL 03	
Colaborador:	Felipe Maia
Localização:	• Arquivo: backend/python/tests/unit/modules/parsers/test_markdown_parser.py • Método: test_parse_file_failure_raises_value_error • Linha: 103 • Commit: refactor(tests): remove Unknown Test in test_parse_file_failure_raise… · fel0psmya/pipeshub-ai@096ab69
Eu estou atualmente trabalhando na refatoração do seguinte test smell:	Unknown Test. O método verifica a exceção com pytest.raises(..., match=...), mas não tem nenhuma instrução assert explícita, e por isso o TEMPY o reporta. Segundo o CATSPY, isso reduz a eficácia dos testes e dificulta a manutenção (categorias: Manutenibilidade e Confiabilidade).
Minhas principais dificuldades na remoção do test smell são:	O exemplo e a refatoração do CATSPY tratam de testes que alteram um estado e passam a verificá-lo com assert, e nenhum cobre testes que esperam uma exceção. Aqui a verificação já existia de forma implícita, via match=, então o desafio foi torná-la explícita sem criar redundância. A sugestão genérica assert exc_info.value is not None foi descartada por ser sempre verdadeira: se o pytest.raises passou, a exceção existe, o que se aproxima do Redundant Assertion do catálogo.
Usei o CATSPY para:	Confirmar a definição e a detecção do Unknown Test (método sem asserções explícitas) e a refatoração prevista (introduzir asserções que validem os resultados esperados). Consultei também a descrição do Redundant Assertion na listagem do catálogo, para não trocar um smell por outro.
Página do CATSPY consultada:	CATSPY - Unknown Test
Eu estou usando os seguintes métodos de refatoração para remover o test smell:	Introdução de asserção explícita, conforme a refatoração do CATSPY para Unknown Test, adaptada ao pytest: a exceção é capturada com as exc_info e a mensagem é validada com assert. O parâmetro match= foi removido para não duplicar a

UNIVERSIDADE FEDERAL DO CEARÁ

	verificação.

SMELL 04	
Colaborador:	Felipe Maia
Localização:	• Arquivo: backend/python/tests/unit/modules/parsers/test_markdown_parser.py • Método: test_parse_file_partial_success_raises • Linha: 111 • Commit: refactor(tests): remove Unknown Test in test_parse_file_partial_succe… · fel0psmya/pipeshub-ai@64599e3
Eu estou atualmente trabalhando na refatoração do seguinte test smell:	Unknown Test. O método verifica a exceção com pytest.raises(..., match=...), mas não tem nenhuma instrução assert explícita, e por isso o TEMPY o reporta. Segundo o CATSPY, isso reduz a eficácia dos testes e dificulta a manutenção (categorias: Manutenibilidade e Confiabilidade).
Minhas principais dificuldades na remoção do test smell são:	O exemplo e a refatoração do CATSPY tratam de testes que alteram um estado e passam a verificá-lo com assert, e nenhum cobre testes que esperam uma exceção. Aqui a verificação já existia de forma implícita, via match=, então o desafio foi torná-la explícita sem criar redundância. A sugestão genérica assert exc_info.value is not None foi descartada por ser sempre verdadeira: se o pytest.raises passou, a exceção existe, o que se aproxima do Redundant Assertion do catálogo.
Usei o CATSPY para:	Confirmar a definição e a detecção do Unknown Test (método sem asserções explícitas) e a refatoração prevista (introduzir asserções que validem os resultados esperados). Consultei também a descrição do Redundant Assertion na listagem do catálogo, para não trocar um smell por outro.
Página do CATSPY consultada:	CATSPY - Unknown Test
Eu estou usando os seguintes métodos de refatoração para remover o test smell:	Introdução de asserção explícita, conforme a refatoração do CATSPY para Unknown Test, adaptada ao pytest: a exceção é capturada com as exc_info e a mensagem é validada com assert. O parâmetro match= foi removido para não duplicar a verificação.

UNIVERSIDADE FEDERAL DO CEARÁ

SMELL 05	
Colaborador:	Felipe Maia
Localização:	• Arquivo: backend/python/tests/unit/modules/parsers/test_markdown_parser.py • Método: test_parse_file_error_status • Linha: 119 • Commit: refactor(tests): remove Unknown Test in test_parse_file_error_status · fel0psmya/pipeshub-ai@3872f95
Eu estou atualmente trabalhando na refatoração do seguinte test smell:	Unknown Test. O método verifica a exceção com pytest.raises(..., match=...), mas não tem nenhuma instrução assert explícita, e por isso o TEMPY o reporta. Segundo o CATSPY, isso reduz a eficácia dos testes e dificulta a manutenção (categorias: Manutenibilidade e Confiabilidade).
Minhas principais dificuldades na remoção do test smell são:	O exemplo e a refatoração do CATSPY tratam de testes que alteram um estado e passam a verificá-lo com assert, e nenhum cobre testes que esperam uma exceção. Aqui a verificação já existia de forma implícita, via match=, então o desafio foi torná-la explícita sem criar redundância. A sugestão genérica assert exc_info.value is not None foi descartada por ser sempre verdadeira: se o pytest.raises passou, a exceção existe, o que se aproxima do Redundant Assertion do catálogo.
Usei o CATSPY para:	Confirmar a definição e a detecção do Unknown Test (método sem asserções explícitas) e a refatoração prevista (introduzir asserções que validem os resultados esperados). Consultei também a descrição do Redundant Assertion na listagem do catálogo, para não trocar um smell por outro.
Página do CATSPY consultada:	CATSPY - Unknown Test
Eu estou usando os seguintes métodos de refatoração para remover o test smell:	Introdução de asserção explícita, conforme a refatoração do CATSPY para Unknown Test, adaptada ao pytest: a exceção é capturada com as exc_info e a mensagem é validada com assert. O parâmetro match= foi removido para não duplicar a verificação.

UNIVERSIDADE
FEDERAL DO CEARÁ

SMELL 06	
Colaborador:	Felipe Maia
Localização:	• Arquivo: backend/python/tests/unit/modules/parsers/test_markdown_parser.py • Método: test_parse_file_empty_status • Linha: 127 • Commit: refactor(tests): remove Unknown Test in test_parse_file_empty_status · fel0psmya/pipeshub-ai@431c8cb
Eu estou atualmente trabalhando na refatoração do seguinte test smell:	Unknown Test. O método verifica a exceção com pytest.raises(..., match=...), mas não tem nenhuma instrução assert explícita, e por isso o TEMPY o reporta. Segundo o CATSPY, isso reduz a eficácia dos testes e dificulta a manutenção (categorias: Manutenibilidade e Confiabilidade).
Minhas principais dificuldades na remoção do test smell são:	O exemplo e a refatoração do CATSPY tratam de testes que alteram um estado e passam a verificá-lo com assert, e nenhum cobre testes que esperam uma exceção. Aqui a verificação já existia de forma implícita, via match=, então o desafio foi torná-la explícita sem criar redundância. A sugestão genérica assert exc_info.value is not None foi descartada por ser sempre verdadeira: se o pytest.raises passou, a exceção existe, o que se aproxima do Redundant Assertion do catálogo.
Usei o CATSPY para:	Confirmar a definição e a detecção do Unknown Test (método sem asserções explícitas) e a refatoração prevista (introduzir asserções que validem os resultados esperados). Consultei também a descrição do Redundant Assertion na listagem do catálogo, para não trocar um smell por outro.
Página do CATSPY consultada:	CATSPY - Unknown Test
Eu estou usando os seguintes métodos de refatoração para remover o test smell:	Introdução de asserção explícita, conforme a refatoração do CATSPY para Unknown Test, adaptada ao pytest: a exceção é capturada com as exc_info e a mensagem é validada com assert. O parâmetro match= foi removido para não duplicar a verificação.

SMELL 07

UNIVERSIDADE FEDERAL DO CEARÁ

Colaborador:	Felipe Maia
Localização:	• Arquivo: backend/python/tests/unit/modules/parsers/test_excel_parser.py • Método: test_formula_cell_data_type_f • Linha: 2800 • Commit: refactor(tests): remove Non-Functional Statement in test_formula_cell… · fel0psmya/pipeshub-ai@1504910
Eu estou atualmente trabalhando na refatoração do seguinte test smell:	Non-Functional Statement. O método monta um mock de célula de fórmula e termina em pass, sem chamar nenhum método sob teste. Segundo o CATSPY, isso gera falso positivo nos resultados do teste e dificulta a manutenção (categorias: Manutenibilidade e Confiabilidade).
Minhas principais dificuldades na remoção do test smell são:	O CATSPY prevê duas refatorações: remover o método ou adicionar a asserção que ele deveria ter. Para escolher, foi preciso descobrir o que o teste pretendia verificar, o que só aparece nos comentários do próprio método: a chave formula adicionada a células com data_type == "f" em _process_sheet. Esse comportamento já é verificado com assert por test_formula_cell_has_formula_key, que usa a mesma célula mock, então completar o teste duplicaria o vizinho. O mesmo método também tinha um Unknown Test, e a remoção resolve os dois em um commit.
Usei o CATSPY para:	Confirmar a definição, os efeitos e as duas refatorações previstas para o smell, e decidir pela remoção em vez de completar o teste.
Página do CATSPY consultada:	CATSPY - Non-Functional Statement
Eu estou usando os seguintes métodos de refatoração para remover o test smell:	Remoção do método de teste sem finalidade, uma das duas opções do catálogo. A alternativa (adicionar a asserção) foi descartada por duplicar test_formula_cell_has_formula_key, o que criaria um teste redundante (cf. Test Redundancy no catálogo).

SMELL 08	
Colaborador:	Felipe Maia
Localização:	• Arquivo:

UNIVERSIDADE FEDERAL DO CEARÁ

	backend/python/tests/unit/modules/parsers/test_excel_parser.py • Método: test_formula_cell_data_type_f • Linha: 2779 • Commit (mesmo do SMELL 07): refactor(tests): remove Non-Functional Statement in test_formula_cell… · fel0psmya/pipeshub-ai@1504910
Eu estou atualmente trabalhando na refatoração do seguinte test smell:	Unknown Test. O método não tem nenhuma asserção e não chama o código sob teste, então passa sempre. Segundo o CATSPY, isso reduz a eficácia dos testes e dificulta a manutenção (categorias: Manutenibilidade e Confiabilidade).
Minhas principais dificuldades na remoção do test smell são:	A refatoração indicada pelo CATSPY para Unknown Test é introduzir asserções explícitas. A asserção que faria sentido (a chave formula no resultado de _process_sheet) já existe em test_formula_cell_has_formula_key, então adicioná-la aqui só criaria redundância. A saída foi remover o método, que é a refatoração do Non-Functional Statement e elimina também esta ocorrência.
Usei o CATSPY para:	Confirmar a definição de Unknown Test e a refatoração prevista, e avaliar se era aplicável a este caso.
Página do CATSPY consultada:	CATSPY - Unknown Test
Eu estou usando os seguintes métodos de refatoração para remover o test smell:	Remoção do método de teste, por não ter asserção nem comportamento próprio a verificar. É a refatoração do Non-Functional Statement, aplicada porque a de Unknown Test (adicionar asserção) duplicaria um teste existente.

SMELL 09	
Colaborador:	Felipe Maia
Localização:	• Arquivo: backend/python/tests/unit/modules/parsers/test_excel_parser.py • Método: test_process_sheet_error_propagates • Linha: 2765 • Commit: refactor(tests): remove Unknown Test in test_process_sheet_error_prop… · fel0psmya/pipeshub-ai@ce46013

UNIVERSIDADE FEDERAL DO CEARÁ

Eu estou atualmente trabalhando na refatoração do seguinte test smell:	Unknown Test. O método verifica a exceção com pytest.raises(RuntimeError, match="sheet error"), mas não tem nenhuma instrução assert explícita, e por isso o TEMPY o reporta. Segundo o CATSPY, isso reduz a eficácia dos testes e dificulta a manutenção (categorias: Manutenibilidade e Confiabilidade).
Minhas principais dificuldades na remoção do test smell são:	O exemplo e a refatoração do CATSPY tratam de testes que alteram um estado e passam a verificá-lo com assert, e nenhum cobre testes que esperam uma exceção. Aqui a verificação já existia de forma implícita, via match=, então o desafio foi torná-la explícita sem criar redundância. A sugestão genérica assert exc_info.value is not None foi descartada por ser sempre verdadeira: se o pytest.raises passou, a exceção existe, o que se aproxima do Redundant Assertion do catálogo. O assert precisou ficar fora do bloco with, porque o pytest.raises interrompe o bloco ao capturar a exceção.
Usei o CATSPY para:	Confirmar a definição e a detecção do Unknown Test (método sem asserções explícitas) e a refatoração prevista (introduzir asserções que validem os resultados esperados). Consultei também a descrição do Redundant Assertion na listagem do catálogo, para não trocar um smell por outro.
Página do CATSPY consultada:	CATSPY - Unknown Test
Eu estou usando os seguintes métodos de refatoração para remover o test smell:	Introdução de asserção explícita, conforme a refatoração do CATSPY para Unknown Test, adaptada ao pytest: a exceção (RuntimeError) é capturada com as exc_info e a mensagem "sheet error" é validada com assert. O parâmetro match= foi removido para não duplicar a verificação.

SMELL 10	
Colaborador:	Felipe Maia
Localização:	• Arquivo: backend/python/tests/unit/modules/parsers/test_excel_parser.py • Método: test_process_cell_error_propagates • Linha: 2917 • Commit: refactor(tests): remove Unknown Test in test_process_cell_error_propa… ·

UNIVERSIDADE FEDERAL DO CEARÁ

	fel0psmya/pipeshub-ai@2d06e84
Eu estou atualmente trabalhando na refatoração do seguinte test smell:	Unknown Test. O método verifica a exceção com pytest.raises(RuntimeError, match="bad cell"), mas não tem nenhuma instrução assert explícita, e por isso o TEMPY o reporta. Segundo o CATSPY, isso reduz a eficácia dos testes e dificulta a manutenção (categorias: Manutenibilidade e Confiabilidade).
Minhas principais dificuldades na remoção do test smell são:	O exemplo e a refatoração do CATSPY tratam de testes que alteram um estado e passam a verificá-lo com assert, e nenhum cobre testes que esperam uma exceção. Aqui a verificação já existia de forma implícita, via match=, então o desafio foi torná-la explícita sem criar redundância. A sugestão genérica assert exc_info.value is not None foi descartada por ser sempre verdadeira: se o pytest.raises passou, a exceção existe, o que se aproxima do Redundant Assertion do catálogo. O assert precisou ficar fora do bloco with, porque o pytest.raises interrompe o bloco ao capturar a exceção.
Usei o CATSPY para:	Confirmar a definição e a detecção do Unknown Test (método sem asserções explícitas) e a refatoração prevista (introduzir asserções que validem os resultados esperados). Consultei também a descrição do Redundant Assertion na listagem do catálogo, para não trocar um smell por outro.
Página do CATSPY consultada:	CATSPY - Unknown Test
Eu estou usando os seguintes métodos de refatoração para remover o test smell:	Introdução de asserção explícita, conforme a refatoração do CATSPY para Unknown Test, adaptada ao pytest: a exceção (RuntimeError) é capturada com as exc_info e a mensagem "bad cell" é validada com assert. O parâmetro match= foi removido para não duplicar a verificação.

SMELL 11	
Colaborador:	Felipe Maia
Localização:	• Arquivo: backend/python/tests/unit/modules/parsers/test_markdown_parser.py • Método: test_protocol_extract_and_replace_images_body • Linha: 332

UNIVERSIDADE FEDERAL DO CEARÁ

	• Commit: refactor(tests): remove Non-Functional Statement in test_protocol_ext… · fel0psmya/pipeshub-ai@4556c32
Eu estou atualmente trabalhando na refatoração do seguinte test smell:	Non-Functional Statement. A instrução pass está no corpo da classe auxiliar _Concrete, definida dentro do teste, e não no método de teste em si. Segundo o CATSPY, esse smell gera falso positivo nos resultados do teste e dificulta a manutenção (categorias: Manutenibilidade e Confiabilidade).
Minhas principais dificuldades na remoção do test smell são:	As refatorações do CATSPY para Non-Functional Statement (remover o método sem finalidade ou adicionar a asserção) não se aplicam literalmente a este caso. O método tem propósito (cobrir o corpo padrão de extract_and_replace_images no protocolo) e já tem asserção (result is None). O pass está numa classe auxiliar que só existe para poder instanciar o protocolo. O TEMPY detecta o pass em qualquer ponto do método, o que aqui parece um falso positivo em relação à definição do catálogo. O desafio foi eliminar a instrução sem alterar o comportamento do teste.
Usei o CATSPY para:	Confirmar a definição, os efeitos e as refatorações previstas para o smell, e verificar se eram aplicáveis, concluindo que o catálogo não cobre exatamente esse caso.
Página do CATSPY consultada:	CATSPY - Non-Functional Statement
Eu estou usando os seguintes métodos de refatoração para remover o test smell:	Substituição do pass por uma docstring na classe auxiliar. É uma adaptação do template do guia, já que o catálogo não prevê o caso. Segue a ideia do CATSPY de eliminar a instrução sem operação funcional e preserva o comportamento do teste e sua asserção.

SMELL 12	
Colaborador:	Felipe Maia
Localização:	• Arquivo: backend/python/tests/unit/modules/parsers/test_remaining_parsers.py • Método: TestDocParser.test_libreoffice_not_installed_raises • Linha: 60 • Commit: refactor(tests): remove Unknown Test in

UNIVERSIDADE FEDERAL DO CEARÁ

	TestDocParser.test_libreoffic… · fel0psmya/pipeshub-ai@fe1a4fe
Eu estou atualmente trabalhando na refatoração do seguinte test smell:	Unknown Test. O método verifica o lançamento de subprocess.CalledProcessError com pytest.raises, mas não tem nenhuma instrução assert explícita. Segundo o CATSPY, isso reduz a eficácia dos testes e dificulta a manutenção (categorias: Manutenibilidade e Confiabilidade).
Minhas principais dificuldades na remoção do test smell são:	Os exemplos do CATSPY tratam de testes que alteram um estado e passam a verificá-lo, e nenhum cobre testes cujo resultado esperado é uma exceção. Além disso, este teste não usa match=, então não havia mensagem a verificar. A asserção escolhida confere o returncode do erro que o próprio teste injeta (side_effect), o que mostra que a exceção chega ao chamador sem ser embrulhada. A limitação é que ela repete o valor injetado. A sugestão genérica assert exc_info.value is not None foi descartada por ser sempre verdadeira, o que se aproxima do Redundant Assertion do catálogo. Outra dificuldade: o mesmo nome de teste existe em três classes do arquivo (TestDocParser, TestXLSParser e TestPPTParser), então cada ocorrência é identificada pela classe e pela linha.
Usei o CATSPY para:	Confirmar a definição e a detecção do Unknown Test e a refatoração prevista (introduzir asserções explícitas), e consultei a descrição do Redundant Assertion para não trocar um smell por outro.
Página do CATSPY consultada:	CATSPY - Unknown Test
Eu estou usando os seguintes métodos de refatoração para remover o test smell:	Introdução de asserção explícita, conforme a refatoração do CATSPY para Unknown Test, adaptada ao pytest: a exceção é capturada com as exc_info e o returncode é validado com assert, posicionado fora do bloco with.

SMELL 13	
Colaborador:	Felipe Maia
Localização:	• Arquivo: backend/python/tests/unit/modules/parsers/test_remaining_parsers.py • Método:

UNIVERSIDADE FEDERAL DO CEARÁ

	TestDocParser.test_conversion_file_not_found_raises • Linha: 73 • Commit: refactor(tests): remove Unknown Test in TestDocParser.test_conversion… · fel0psmya/pipeshub-ai@fa59a17
Eu estou atualmente trabalhando na refatoração do seguinte test smell:	Unknown Test. O método verifica o lançamento de uma exceção com pytest.raises(..., match="output file not found"), mas não tem nenhuma instrução assert explícita. Segundo o CATSPY, isso reduz a eficácia dos testes e dificulta a manutenção (categorias: Manutenibilidade e Confiabilidade).
Minhas principais dificuldades na remoção do test smell são:	O catálogo não traz exemplos de testes que esperam uma exceção, então foi preciso adaptar a refatoração. A verificação já existia de forma implícita, via match=, e o desafio foi torná-la explícita sem duplicá-la: o match= foi removido e a mensagem passou a ser checada com assert. O assert precisou ficar fora do pytest.raises (que interrompe o bloco ao lançar a exceção) e fora dos patch(...), que mockam open e os.path.exists. A sugestão genérica assert exc_info.value is not None foi descartada por ser sempre verdadeira. A docstring menciona FileNotFoundError, mas o teste aceita qualquer Exception, e mantive isso para não alterar o comportamento. Também há o mesmo nome de teste em outras duas classes do arquivo, então a identificação é pela classe e pela linha.
Usei o CATSPY para:	Confirmar a definição e a detecção do Unknown Test e a refatoração prevista, e consultei o Redundant Assertion para evitar uma asserção sem valor.
Página do CATSPY consultada:	CATSPY - Unknown Test
Eu estou usando os seguintes métodos de refatoração para remover o test smell:	Introdução de asserção explícita, conforme o CATSPY, adaptada ao pytest: a exceção é capturada com as exc_info e a mensagem é validada com assert ... in str(exc_info.value), equivalente ao match= original.

SMELL 14	
Colaborador:	Felipe Maia

UNIVERSIDADE FEDERAL DO CEARÁ

Localização:	• Arquivo: backend/python/tests/unit/modules/agents/context/test_tool_descriptions.py • Método: test_annotation_without_origin_or_name • Linha: 95 • Commit: refactor(tests): remove Non-Functional Statement in test_annotation_w… · fel0psmya/pipeshub-ai@2afd04c
Eu estou atualmente trabalhando na refatoração do seguinte test smell:	Non-Functional Statement. O método define uma classe auxiliar interna (_Weird) cujo corpo é apenas a instrução pass. Segundo o CATSPY, instruções sem operação funcional geram falso positivo nos resultados e dificultam a manutenção (categorias: Manutenibilidade e Confiabilidade).
Minhas principais dificuldades na remoção do test smell são:	As duas refatorações do CATSPY (remover o método sem finalidade ou adicionar a asserção que ele deveria ter) não se aplicam literalmente. O método tem propósito e já tem asserção, e o pass está numa classe auxiliar que só existe para servir de anotação de tipo no teste. O TEMPY detecta o pass em qualquer ponto do método, o que aqui parece um falso positivo em relação à definição do catálogo. O desafio foi eliminar a instrução sem alterar o comportamento do teste.
Usei o CATSPY para:	Confirmar a definição, os efeitos e as refatorações previstas para o smell e verificar se eram aplicáveis, concluindo que o catálogo não cobre exatamente esse caso.
Página do CATSPY consultada:	CATSPY - Non-Functional Statement
Eu estou usando os seguintes métodos de refatoração para remover o test smell:	Substituição do pass por uma docstring na classe auxiliar. É uma adaptação do template do guia, já que o catálogo não prevê o caso. Segue a ideia do CATSPY de eliminar a instrução sem operação funcional e preserva o comportamento e a asserção do teste.

SMELL 15	
Colaborador:	Felipe Maia
Localização:	• Arquivo: backend/python/tests/unit/modules/indexing/test_rate_limit_deferral.py • Método: test_found_under_a_wrapper

UNIVERSIDADE FEDERAL DO CEARÁ

	• Linha: 39 • Commit: refactor(tests): remove Exception Handling in test_found_under_a_wrap… · fel0psmya/pipeshub-ai@222a1b8
Eu estou atualmente trabalhando na refatoração do seguinte test smell:	Exception Handling. O método usa um bloco try/except externo para capturar o RuntimeError lançado e só então verificar o valor com assert, em vez de usar os recursos do framework de testes. Segundo o CATSPY, isso ofusca a intenção do teste e reduz a confiabilidade e a clareza (categorias: Legibilidade e Confiabilidade).
Minhas principais dificuldades na remoção do test smell são:	O método tem dois blocos try aninhados, que o TEMPY reporta como duas ocorrências (linhas 39 e 40), e cada um serve a um propósito diferente. O externo é o caso do catálogo, porque captura a exceção só para verificá-la, e a refatoração do CATSPY (assertRaises) corresponde ao pytest.raises. A dificuldade foi manter a asserção sobre a exceção capturada, que passou a ser feita depois do bloco with, já que o pytest.raises interrompe o bloco no ponto em que a exceção é lançada. O try interno foi tratado em um commit separado (SMELL 16).
Usei o CATSPY para:	Confirmar a definição e a detecção do Exception Handling (método de teste que usa blocos try-except) e a refatoração prevista, que é usar assertRaises em vez de capturar a exceção manualmente.
Página do CATSPY consultada:	CATSPY - Exception Handling
Eu estou usando os seguintes métodos de refatoração para remover o test smell:	Substituição do try/except externo por with pytest.raises(RuntimeError) as exc_info, equivalente em pytest ao assertRaises indicado pelo CATSPY, com a asserção sobre exc_info.value feita após o bloco.

SMELL 16	
Colaborador:	Felipe Maia
Localização:	• Arquivo: backend/python/tests/unit/modules/indexing/test_rate_limit_deferral.py • Método: test_found_under_a_wrapper • Linha: 40

	• Commit: refactor(tests): remove Exception Handling in test_found_under_a_wrap… · fel0psmya/pipeshub-ai@0201b31
Eu estou atualmente trabalhando na refatoração do seguinte test smell:	Exception Handling. O bloco try/except interno do método existia apenas para construir uma cadeia de exceções (uma RuntimeError lançada a partir de uma ApiCallError, com raise ... from), e não para verificar um resultado. Segundo o CATSPY, o smell reduz a clareza e a confiabilidade do teste (categorias: Legibilidade e Confiabilidade).
Minhas principais dificuldades na remoção do test smell são:	Este caso não é o descrito no catálogo: o try não captura a exceção para testá-la, só monta a causa do erro que será embrulhado, então a refatoração do CATSPY (assertRaises) não se aplica a esse bloco. O desafio foi montar a cadeia sem capturar nada e preservar o comportamento. Foi preciso confirmar que o teste continuava passando com a cadeia montada por raise ... from, já que, fora de um except, só a causa explícita fica preenchida e o contexto implícito não, e isso poderia afetar a função que lê a cadeia.
Usei o CATSPY para:	Confirmar a definição do Exception Handling e a refatoração prevista, e verificar se era aplicável a um try usado para construir uma exceção, concluindo que o catálogo não cobre esse uso.
Página do CATSPY consultada:	CATSPY - Exception Handling
Eu estou usando os seguintes métodos de refatoração para remover o test smell:	Remoção do try/except interno: a exceção de origem (_limited(600)) é criada diretamente e a RuntimeError é lançada com raise ... from dentro de with pytest.raises. É uma adaptação ao pytest, já que o catálogo só cobre try/except usado para verificar exceções.

SMELL 17	
Colaborador:	Felipe Maia
Localização:	• Arquivo: backend/python/tests/unit/modules/parsers/test_markdown_to_blocks.py • Método: _child_block_indices • Linha: 44

	• Commit: refactor(tests): remove Conditional Test Logic in _child_block_indices · fel0psmya/pipeshub-ai@95a0bab
Eu estou atualmente trabalhando na refatoração do seguinte test smell:	Conditional Test Logic. O helper _child_block_indices contém um laço for (percorre os intervalos de blocos de um grupo) além de uma asserção, e por isso o TEMPY o reporta. Segundo o CATSPY, instruções condicionais ou laços em métodos de teste afetam a compreensão e a manutenção (categorias: Legibilidade, Manutenibilidade e Confiabilidade).
Minhas principais dificuldades na remoção do test smell são:	O alvo não é um método de teste, e sim um helper que precisa percorrer uma lista de intervalos. A refatoração do CATSPY (separar cada caso condicional em um método de teste próprio) não pode ser aplicada, porque aqui não há casos a separar. O aviso é um falso positivo em relação à definição do catálogo, e a solução foi uma adaptação: expressar a iteração como uma compreensão de lista. O assert de guarda foi mantido neste commit, e o Undefined Test do mesmo helper foi tratado no commit seguinte (SMELL 18), para manter um smell por commit.
Usei o CATSPY para:	Confirmar a definição e a detecção do Conditional Test Logic e a refatoração prevista (extrair os casos condicionais para métodos de teste separados), e avaliar se era aplicável a um helper.
Página do CATSPY consultada:	CATSPY - Conditional Test Logic
Eu estou usando os seguintes métodos de refatoração para remover o test smell:	Substituição do laço for por uma list comprehension equivalente, mantendo o resultado e o assert de guarda. É uma adaptação, já que a refatoração do catálogo (um teste por caso) não se aplica a helpers.

SMELL 18	
Colaborador:	Felipe Maia
Localização:	• Arquivo: backend/python/tests/unit/modules/parsers/test_markdown_to_blocks.py • Método: _child_block_indices • Linha: 41

UNIVERSIDADE FEDERAL DO CEARÁ

	• Commit: refactor(tests): remove Undefined Test in _child_block_indices · fel0psmya/pipeshub-ai@732c3cd
Eu estou atualmente trabalhando na refatoração do seguinte test smell:	Undefined Test. _child_block_indices é uma função auxiliar cujo nome não começa com test e que contém uma asserção (o assert de guarda sobre group.children), e o TEMPY a reporta como um método de teste que o framework não reconhece. Segundo o CATSPY, o smell pode passar a impressão de que está tudo correto quando o método não foi avaliado (categorias: Manutenibilidade e Confiabilidade).
Minhas principais dificuldades na remoção do test smell são:	As duas refatorações do CATSPY (renomear com o prefixo test ou remover o método) não servem: o helper é usado por outros testes, e renomeá-lo faria o pytest coletá-lo como teste e falhar por fixture inexistente (group), enquanto removê-lo quebraria os chamadores. A sugestão do guia (mover para fora da classe) também não se aplica, porque a função já está no nível do módulo. O aviso é um falso positivo em relação ao catálogo, já que o helper nunca deveria ser executado como teste. Pelo código-fonte do TEMPY, o detector marca funções sem prefixo test que contêm uma asserção, então o ajuste foi retirar o assert de guarda, que servia só para estreitar o tipo. A consequência é que, se children for None, a falha passa de AssertionError para AttributeError no teste que chamou o helper.
Usei o CATSPY para:	Confirmar a definição e a detecção do Undefined Test (método sem o prefixo test) e as refatorações previstas (renomear ou remover), concluindo que não eram aplicáveis a um helper.
Página do CATSPY consultada:	CATSPY - Undefined Test
Eu estou usando os seguintes métodos de refatoração para remover o test smell:	Remoção da asserção de guarda do helper. É uma adaptação, pois o catálogo só prevê renomear ou remover o método. O helper continua devolvendo o mesmo resultado.

SMELL 19	
Colaborador:	Felipe Maia

UNIVERSIDADE FEDERAL DO CEARÁ

Localização:	• Arquivo: backend/python/tests/unit/modules/parsers/test_markdown_to_blocks.py • Método: _child_group_indices • Linha: 52 • Commit: refactor(tests): remove Conditional Test Logic in _child_group_indices · fel0psmya/pipeshub-ai@1f5b7e4
Eu estou atualmente trabalhando na refatoração do seguinte test smell:	Conditional Test Logic. O helper _child_group_indices contém um laço for (percorre os intervalos de grupos de blocos) além de uma asserção, e por isso o TEMPY o reporta. Segundo o CATSPY, instruções condicionais ou laços em métodos de teste afetam a compreensão e a manutenção (categorias: Legibilidade, Manutenibilidade e Confiabilidade).
Minhas principais dificuldades na remoção do test smell são:	O alvo não é um método de teste, e sim um helper que precisa percorrer uma lista de intervalos. A refatoração do CATSPY (separar cada caso condicional em um método de teste próprio) não pode ser aplicada, porque aqui não há casos a separar. O aviso é um falso positivo em relação à definição do catálogo, e a solução foi uma adaptação: expressar a iteração como uma compreensão de lista. O assert de guarda foi mantido neste commit, e o Undefined Test do mesmo helper foi tratado no commit seguinte (SMELL 20), para manter um smell por commit.
Usei o CATSPY para:	Confirmar a definição e a detecção do Conditional Test Logic e a refatoração prevista (extrair os casos condicionais para métodos de teste separados), e avaliar se era aplicável a um helper.
Página do CATSPY consultada:	CATSPY - Conditional Test Logic
Eu estou usando os seguintes métodos de refatoração para remover o test smell:	Substituição do laço for por uma list comprehension equivalente, mantendo o resultado e o assert de guarda. É uma adaptação, já que a refatoração do catálogo (um teste por caso) não se aplica a helpers.

SMELL 20	
Colaborador:	Felipe Maia

UNIVERSIDADE FEDERAL DO CEARÁ

Localização:	• Arquivo: backend/python/tests/unit/modules/parsers/test_markdown_to_blocks.py • Método: _child_group_indices • Linha: 49 • Commit: refactor(tests): remove Undefined Test in _child_group_indices · fel0psmya/pipeshub-ai@a6f5a8c
Eu estou atualmente trabalhando na refatoração do seguinte test smell:	Undefined Test. _child_group_indices é uma função auxiliar cujo nome não começa com test e que contém uma asserção (o assert de guarda sobre group.children), e o TEMPY a reporta como um método de teste que o framework não reconhece. Segundo o CATSPY, o smell pode passar a impressão de que está tudo correto quando o método não foi avaliado (categorias: Manutenibilidade e Confiabilidade).
Minhas principais dificuldades na remoção do test smell são:	As duas refatorações do CATSPY (renomear com o prefixo test ou remover o método) não servem: o helper é usado por outros testes, e renomeá-lo faria o pytest coletá-lo como teste e falhar por fixture inexistente (group), enquanto removê-lo quebraria os chamadores. A sugestão do guia (mover para fora da classe) também não se aplica, porque a função já está no nível do módulo. O aviso é um falso positivo em relação ao catálogo, já que o helper nunca deveria ser executado como teste. Pelo código-fonte do TEMPY, o detector marca funções sem prefixo test que contêm uma asserção, então o ajuste foi retirar o assert de guarda, que servia só para estreitar o tipo. A consequência é que, se children for None, a falha passa de AssertionError para AttributeError no teste que chamou o helper.
Usei o CATSPY para:	Confirmar a definição e a detecção do Undefined Test (método sem o prefixo test) e as refatorações previstas (renomear ou remover), concluindo que não eram aplicáveis a um helper.
Página do CATSPY consultada:	CATSPY - Undefined Test
Eu estou usando os seguintes métodos de refatoração para remover o test smell:	Remoção da asserção de guarda do helper. É uma adaptação, pois o catálogo só prevê renomear ou remover o método. O helper continua devolvendo o mesmo resultado.

UNIVERSIDADE
FEDERAL DO CEARÁ

SMELL 21	
Colaborador:	Felipe Maia
Localização:	• Arquivo: backend/python/tests/unit/modules/parsers/test_markdown_to_blocks.py • Método: _assert_all_blocks_have_data • Linha: 72 • Commit: refactor(tests): remove Conditional Test Logic in _assert_all_blocks_… · fel0psmya/pipeshub-ai@f593777
Eu estou atualmente trabalhando na refatoração do seguinte test smell:	Conditional Test Logic. O helper _assert_all_blocks_have_data percorre todos os blocos do container com um laço for e faz uma asserção dentro dele. Segundo o CATSPY, laços em métodos de teste afetam a compreensão e a manutenção (categorias: Legibilidade, Manutenibilidade e Confiabilidade).
Minhas principais dificuldades na remoção do test smell são:	O alvo é um helper de asserção que precisa verificar todos os blocos, então a refatoração do CATSPY (separar os casos em testes próprios) não se aplica, e o aviso é um falso positivo em relação ao catálogo. A solução foi uma adaptação: reunir os blocos sem data numa compreensão de lista e fazer uma única asserção sobre ela. Isso muda a mensagem de falha, que agora lista todos os blocos com data=None e não só o primeiro, mas o critério de passar ou falhar é o mesmo.
Usei o CATSPY para:	Confirmar a definição e a detecção do Conditional Test Logic e a refatoração prevista (extrair os casos condicionais para métodos de teste separados), e avaliar se era aplicável a um helper.
Página do CATSPY consultada:	CATSPY - Conditional Test Logic
Eu estou usando os seguintes métodos de refatoração para remover o test smell:	Substituição do laço for com asserção interna por uma list comprehension que reúne os blocos sem data e por uma única asserção sobre essa lista. É uma adaptação, já que o catálogo prevê separar casos condicionais em testes distintos.

SMELL 22	
Colaborador:	Felipe Maia

UNIVERSIDADE FEDERAL DO CEARÁ

Localização:	• Arquivo: backend/python/tests/unit/modules/parsers/test_opencv_layout_analyzer_cov95.py • Método: test_render_all_pages • Linha: 1266 • Commit: refactor(tests): remove Exception Handling in test_render_all_pages · fel0psmya/pipeshub-ai@cb5c9d7
Eu estou atualmente trabalhando na refatoração do seguinte test smell:	Exception Handling. O método usa um bloco try/finally para apagar o arquivo PDF temporário que ele mesmo cria. Segundo o CATSPY, o smell ocorre quando um método de teste usa blocos try-except (categorias: Legibilidade e Confiabilidade).
Minhas principais dificuldades na remoção do test smell são:	O try do método não captura nenhuma exceção para decidir o resultado do teste, é uma limpeza de recurso (try/finally). O caso descrito no CATSPY (try/except com fail, substituído por assertRaises) não é este, então o aviso do TEMPY é um falso positivo em relação ao catálogo e a refatoração do catálogo não se aplica. Foi preciso manter a limpeza do arquivo sem usar try. O teste também chama pytest.importorskip("reportlab"), então ele só executa se a biblioteca estiver instalada, e por isso foi preciso conferir que ele roda de fato e não é pulado.
Usei o CATSPY para:	Confirmar a definição e a detecção do Exception Handling (blocos try-except em métodos de teste) e a refatoração prevista (assertRaises), concluindo que o catálogo não cobre o uso de try/finally para limpeza de recursos.
Página do CATSPY consultada:	CATSPY - Exception Handling
Eu estou usando os seguintes métodos de refatoração para remover o test smell:	Substituição do arquivo temporário criado à mão e do try/finally pelo fixture tmp_path do pytest, que cria e remove o diretório sozinho. Os imports locais dos módulos tempfile e os deixaram de ser necessários. É uma adaptação ao pytest, já que o catálogo não prevê esse caso.

SMELL 23	
Colaborador:	Felipe Maia

Localização:	• Arquivo: backend/python/tests/unit/modules/parsers/test_opencv_layout_analyzer_cov95.py • Método: test_rasterize_page_with_pdf_path • Linha: 1679 • Commit: refactor(tests): remove Exception Handling in test_rasterize_page_wit… · fel0psmya/pipeshub-ai@68d1185
Eu estou atualmente trabalhando na refatoração do seguinte test smell:	Exception Handling. O método gera um PDF temporário, usa o caminho dele no teste de _rasterize_page e o apaga num bloco try/finally. Segundo o CATSPY, o smell ocorre quando um método de teste usa blocos try-except (categorias: Legibilidade e Confiabilidade).
Minhas principais dificuldades na remoção do test smell são:	O try do método não captura nenhuma exceção para decidir o resultado do teste, é uma limpeza de recurso (try/finally). O caso descrito no CATSPY (try/except com fail, substituído por assertRaises) não é este, então o aviso do TEMPY é um falso positivo em relação ao catálogo e a refatoração do catálogo não se aplica. O desafio foi manter a limpeza do arquivo sem try e garantir que o teste continuasse recebendo o caminho do PDF como texto, como antes.
Usei o CATSPY para:	Confirmar a definição e a detecção do Exception Handling (blocos try-except em métodos de teste) e a refatoração prevista (assertRaises), concluindo que o catálogo não cobre o uso de try/finally para limpeza de recursos.
Página do CATSPY consultada:	CATSPY - Exception Handling
Eu estou usando os seguintes métodos de refatoração para remover o test smell:	Substituição do arquivo temporário criado à mão e do try/finally pelo fixture tmp_path do pytest, que cria e remove o diretório sozinho. Os imports locais dos módulos tempfile e os deixaram de ser necessários. É uma adaptação ao pytest, já que o catálogo não prevê esse caso.

SMELL 24	
Colaborador:	Felipe Maia
Localização:	• Arquivo: backend/python/tests/unit/modules/parsers/test_excel_parser.py • Método: test_values_are_strings

	• Linha: 389 • Commit: refactor(tests): remove Conditional Test Logic in test_values_are_str… · fel0psmya/pipeshub-ai@cf416b7
Eu estou atualmente trabalhando na refatoração do seguinte test smell:	Conditional Test Logic. O método percorre com um laço for todas as entradas de BUILTIN_DATE_FORMATS e verifica, para cada uma, se o formato é uma string. Segundo o CATSPY, laços e condicionais em métodos de teste afetam a compreensão e a manutenção (categorias: Legibilidade, Manutenibilidade e Confiabilidade).
Minhas principais dificuldades na remoção do test smell são:	O catálogo exemplifica a refatoração separando cada ramo de um if/else em um método de teste próprio. Aqui o laço percorre uma constante, e escrever um método por entrada duplicaria código e ficaria desatualizado se a constante mudasse. A solução foi usar o mecanismo equivalente do pytest, que gera um caso de teste por entrada. O efeito colateral é que um teste passou a ser vários casos (9, um por código de formato), então o número de testes coletados aumenta. Outro ponto é que, se a constante ficasse vazia, o teste seria pulado em vez de passar sem verificar nada, o que não ocorre aqui.
Usei o CATSPY para:	Confirmar a definição e a detecção do Conditional Test Logic e a refatoração prevista (extrair os casos condicionais para métodos de teste separados), para escolher a forma de aplicá-la a um laço sobre uma constante.
Página do CATSPY consultada:	CATSPY - Conditional Test Logic
Eu estou usando os seguintes métodos de refatoração para remover o test smell:	Substituição do laço for por @pytest.mark.parametrize sobre BUILTIN_DATE_FORMATS.items(), gerando um caso de teste por entrada. É a tradução para pytest da refatoração do CATSPY (um teste por caso).

SMELL 25	
Colaborador:	Felipe Maia
Localização:	• Arquivo: backend/python/tests/unit/modules/parsers/test_excel_parser.py • Método: test_all_entries_are_tuples

UNIVERSIDADE FEDERAL DO CEARÁ

	• Linha: 398 • Commit: refactor(tests): remove Conditional Test Logic in test_all_entries_ar… · fel0psmya/pipeshub-ai@3c047e4
Eu estou atualmente trabalhando na refatoração do seguinte test smell:	Conditional Test Logic. O método percorre com um laço for todas as entradas de COMMON_FORMAT_WHITELIST e verifica, para cada uma, se o valor é uma tupla com exatamente dois elementos. Segundo o CATSPY, laços e condicionais em métodos de teste afetam a compreensão e a manutenção (categorias: Legibilidade, Manutenibilidade e Confiabilidade).
Minhas principais dificuldades na remoção do test smell são:	O catálogo exemplifica a refatoração separando cada ramo de um if/else em um método de teste próprio. Aqui o laço percorre uma constante e faz duas asserções por entrada, e escrever um método por entrada duplicaria código e ficaria desatualizado se a constante mudasse. A solução foi usar o mecanismo equivalente do pytest, que gera um caso de teste por entrada – nesse caso, 35 casos de teste – e mantém as duas asserções, com as mesmas mensagens, em cada caso. O efeito colateral é que um teste passou a ser um caso por entrada da whitelist, então o número de testes coletados aumenta.
Usei o CATSPY para:	Confirmar a definição e a detecção do Conditional Test Logic e a refatoração prevista (extrair os casos condicionais para métodos de teste separados), para escolher a forma de aplicá-la a um laço sobre uma constante.
Página do CATSPY consultada:	CATSPY - Conditional Test Logic
Eu estou usando os seguintes métodos de refatoração para remover o test smell:	Substituição do laço for por @pytest.mark.parametrize sobre COMMON_FORMAT_WHITELIST.items(), gerando um caso de teste por entrada. É a tradução para pytest da refatoração do CATSPY (um teste por caso).

SMELL 26	
Colaborador:	Guilherme Sousa
Localização:	• Arquivo: backend/python/tests/unit/modules/parsers/test_html_parser_shim.py

	• Método: test_protocol_clean_html_body • Linha: 16 • Commit: refactor(tests): remove Non-Functional Statement in test_protocol_cle… · fel0psmya/pipeshub-ai@88ed089
Eu estou atualmente trabalhando na refatoração do seguinte test smell:	Non-Functional Statement. A classe auxiliar _Concrete (subclasse de HTMLParserProtocol), declarada dentro do método de teste, tem como corpo apenas a instrução pass.
Minhas principais dificuldades na remoção do test smell são:	O teste já tem asserção, e o pass está numa classe auxiliar que precisa continuar vazia para herdar os corpos padrão do Protocol. Como o padrão se repete em três testes do arquivo, mantive a mesma solução em todos, mas cada ocorrência foi corrigida em um commit separado, conforme a regra de 1 commit por smell.
Usei o CATSPY para:	Confirmar que a ocorrência se enquadra em Non-Functional Statement e manter a refatoração coerente com a adotada no SMELL 11.
Página do CATSPY consultada:	CATSPY - Non-Functional Statement
Eu estou usando os seguintes métodos de refatoração para remover o test smell:	Substituição da instrução sem efeito (pass) por uma docstring que documenta a intenção da classe auxiliar. O comportamento do teste não mudou e o arquivo passa no pytest (7 passed).

SMELL 27	
Colaborador:	Guilherme Sousa
Localização:	• Arquivo: backend/python/tests/unit/modules/parsers/test_html_parser_shim.py • Método: test_protocol_replace_relative_image_urls_body • Linha: 24 • Commit: refactor(tests): remove Non-Functional Statement in test_protocol_extract_and_replace_images_body · fel0psmya/pipeshub-ai@a43fcc9
Eu estou atualmente trabalhando na	Non-Functional Statement. A classe auxiliar _Concrete

refatoração do seguinte test smell:	(subclasse de HTMLParserProtocol), declarada dentro do método de teste, tem como corpo apenas a instrução pass.
Minhas principais dificuldades na remoção do test smell são:	É a segunda ocorrência do mesmo padrão no arquivo (ver SMELL 26). O teste já tem asserção, e a classe precisa continuar vazia para herdar o corpo padrão de replace_relative_image_urls do Protocol. O ponto de atenção foi manter a solução idêntica nas três ocorrências, para o arquivo continuar consistente, mas com cada uma em um commit separado.
Usei o CATSPY para:	Confirmar que a ocorrência se enquadra em Non-Functional Statement e aplicar a mesma refatoração do SMELL 26.
Página do CATSPY consultada:	CATSPY - Non-Functional Statement
Eu estou usando os seguintes métodos de refatoração para remover o test smell:	Substituição da instrução sem efeito (pass) por uma docstring que documenta a intenção da classe auxiliar. O comportamento do teste não mudou e o arquivo passa no pytest (7 passed).

SMELL 28	
Colaborador:	Guilherme Sousa
Localização:	• Arquivo: backend/python/tests/unit/modules/parsers/test_html_parser_shim.py • Método: test_protocol_extract_and_replace_images_body • Linha: 32 • Commit: refactor(tests): remove Non-Functional Statement in test_protocol_extract_and_replace_images_body · fel0psmya/pipeshub-ai@c43e006
Eu estou atualmente trabalhando na refatoração do seguinte test smell:	Non-Functional Statement. A classe auxiliar _Concrete (subclasse de HTMLParserProtocol), declarada dentro do método de teste, tem como corpo apenas a instrução pass.
Minhas principais dificuldades na remoção do test smell são:	É a terceira ocorrência do mesmo padrão no arquivo (ver SMELLS 26 e 27). O teste já tem asserção, e a classe precisa continuar vazia para herdar o corpo padrão de extract_and_replace_images do Protocol. O ponto de

	atenção foi manter a solução idêntica nas três ocorrências, para o arquivo continuar consistente, mas com cada uma em um commit separado.
Usei o CATSPY para:	Confirmar que a ocorrência se enquadra em Non-Functional Statement e aplicar a mesma refatoração dos SMELLS 26 e 27.
Página do CATSPY consultada:	CATSPY - Non-Functional Statement
Eu estou usando os seguintes métodos de refatoração para remover o test smell:	Substituição da instrução sem efeito (pass) por uma docstring que documenta a intenção da classe auxiliar. O comportamento do teste não mudou e o arquivo passa no pytest (7 passed).

SMELL 29	
Colaborador:	Guilherme Sousa
Localização:	• Arquivo: backend/python/tests/unit/modules/parsers/test_pdf_rasterizer.py • Método: test_broken_process_pool_reports_memory_incident_when_governor_wired • Linha: 209 • Commit: refactor(tests): remove Exception Handling in test_broken_process_pool_reports_memory_incident_when_governor_wired · fel0psmya/pipeshub-ai@d7d9f40
Eu estou atualmente trabalhando na refatoração do seguinte test smell:	Exception Handling. O método de teste usa um bloco try/finally manual para garantir que o ResourceGovernor mock, ligado ao módulo pdf_rasterizer por uma variável global, seja desligado no fim do teste. O controle de fluxo manual mistura a preparação do ambiente com a verificação e esconde a intenção do teste.
Minhas principais dificuldades na remoção do test smell são:	O exemplo do CATSPY trata de um try/except usado para verificar uma exceção, e esse teste já fazia essa verificação

	com pytest.raises. O try existia para outra coisa: limpar estado global. Por isso não bastava apagar o bloco. Sem o finally, o governor ficaria ligado e quebraria o teste seguinte (test_broken_process_pool_without_governor_does_not_raise), que espera _resource_governor is None. Foi preciso achar no pytest um recurso nativo que garantisse a limpeza mesmo quando o teste falha.
Usei o CATSPY para:	Entender a definição de Exception Handling e o princípio da refatoração proposta: substituir o tratamento manual com try pelos recursos nativos do framework de teste. Adaptei esse princípio ao caso, em que o try servia para preparar e limpar o ambiente, e não para conferir uma exceção.
Página do CATSPY consultada:	CATSPY - Exception Handling
Eu estou usando os seguintes métodos de refatoração para remover o test smell:	Substituição do bloco try/finally por uma fixture do pytest com yield (wired_governor). A fixture liga o governor mock antes do teste, entrega o mock ao teste e o desliga depois do yield, e o pytest garante essa limpeza mesmo em caso de falha. O corpo do teste ficou só com a preparação, a verificação da exceção (pytest.raises) e a asserção. O arquivo passa no pytest (15 passed), e o TEMPY deixou de apontar o smell.

SMELL 30	
Colaborador:	Guilherme Sousa
Localização:	• Arquivo: backend/python/tests/unit/modules/parsers/test_pdf_rasterizer.py • Método: test_effective_resolution_leaves_standard_pages_untouched • Linha: 116 • Commit: refactor(tests): remove Conditional Test Logic in test_effective_resolution_leaves_standard_pages_untouched · fel0psmya/pipeshub-ai@814eaaa

UNIVERSIDADE FEDERAL DO CEARÁ

Eu estou atualmente trabalhando na refatoração do seguinte test smell:	Conditional Test Logic. O método de teste usa um laço for para percorrer três tamanhos de página padrão (A4, Letter e A2) e faz a asserção dentro do laço. Isso junta três casos de teste em um só: se um tamanho falhar, o relatório não mostra qual foi, e os casos seguintes nem chegam a ser executados.
Minhas principais dificuldades na remoção do test smell são:	O CATSPY recomenda separar cada caso em um método de teste próprio. Escrever três funções quase idênticas, porém, duplicaria o mesmo corpo três vezes e criaria outro problema de manutenção. A dificuldade foi aplicar a separação de casos sem duplicar código e mantendo os mesmos valores de entrada do teste original.
Usei o CATSPY para:	Confirmar a definição de Conditional Test Logic (teste com estruturas como if, for ou while) e seguir a refatoração proposta: separar cada caso em um teste independente, como no exemplo em que test_is_adult vira test_is_adult_true e test_is_adult_false.
Página do CATSPY consultada:	CATSPY - Conditional Test Logic
Eu estou usando os seguintes métodos de refatoração para remover o test smell:	Separação dos casos de teste com @pytest.mark.parametrize, o recurso do pytest para gerar um teste independente por caso sem duplicar o corpo. O laço for foi removido, e cada tamanho de página virou um caso nomeado (a4, letter, a2), que aparece e falha separadamente no relatório. O arquivo passa no pytest (17 passed, eram 15, porque o teste virou 3 casos), e o TEMPY deixou de apontar o smell.

SMELL 31	
Colaborador:	Guilherme Sousa
Localização:	• Arquivo: backend/python/tests/unit/modules/agents/qna/test_schemas.py • Método: test_all_confidence_values • Linha: 93 • Commit: refactor(tests): remove Conditional Test Logic in test_all_confidence_values · fel0psmya/pipeshub-ai@32dcf6d

UNIVERSIDADE FEDERAL DO CEARÁ

Eu estou atualmente trabalhando na refatoração do seguinte test smell:	Conditional Test Logic. O método de teste usa um laço for para percorrer os quatro níveis de confiança aceitos pelo modelo AgentAnswerWithMetadataJSON (“Very High”, “High”, “Medium”, “Low”) e faz a asserção dentro do laço. São quatro casos de teste dentro de um único método: se um nível falhar, o relatório não mostra qual foi, e os níveis seguintes deixam de ser verificados.
Minhas principais dificuldades na remoção do test smell são:	Como no SMELL 30, separar os casos em quatro métodos escritos à mão repetiria o mesmo corpo quatro vezes. Além disso, este teste é um método dentro de uma classe (TestAgentAnswerWithMetadataJSON), então foi preciso conferir que a parametrização funciona em métodos de classe e que o parâmetro entra depois do self na assinatura.
Usei o CATSPY para:	Confirmar que o laço for dentro do teste caracteriza Conditional Test Logic e seguir a refatoração proposta: separar cada caso em um teste independente.
Página do CATSPY consultada:	CATSPY - Conditional Test Logic
Eu estou usando os seguintes métodos de refatoração para remover o test smell:	Separação dos casos de teste com @pytest.mark.parametrize("level", [...]): o laço for foi removido e cada nível de confiança passou a ser um caso de teste independente, que aparece e falha separadamente no relatório. A asserção e os dados de entrada são os mesmos do teste original. O arquivo passa no pytest (21 passed), e o TEMPY deixou de apontar o smell.

SMELL 32	
Colaborador:	Guilherme Sousa
Localização:	• Arquivo: backend/python/tests/unit/modules/agents/qna/test_schemas.py • Método: test_all_answer_match_types • Linha: 109 • Commit: refactor(tests): remove Conditional Test Logic in test_all_answer_match_types · fel0psmya/pipeshub-ai@7037c31

UNIVERSIDADE FEDERAL DO CEARÁ

Eu estou atualmente trabalhando na refatoração do seguinte test smell:	Conditional Test Logic. O método de teste monta uma lista com os cinco tipos de correspondência aceitos pelo campo answerMatchType do modelo AgentAnswerWithMetadataJSON e usa um laço for para criar e verificar uma instância para cada tipo. São cinco casos de teste em um único método: se um tipo falhar, os seguintes não são verificados e o relatório não mostra qual tipo falhou.
Minhas principais dificuldades na remoção do test smell são:	A lista de tipos era uma variável local (valid_types) usada só pelo laço. Para separar os casos, os dados tiveram de sair do corpo do teste e ir para o decorator. A cautela foi conferir que os cinco tipos continuassem exatamente os mesmos e na mesma ordem, para a cobertura do teste não mudar.
Usei o CATSPY para:	Confirmar que o laço for dentro do teste caracteriza Conditional Test Logic e aplicar a refatoração proposta, separando cada caso em um teste independente, como nos SMELLS 30 e 31.
Página do CATSPY consultada:	CATSPY - Conditional Test Logic
Eu estou usando os seguintes métodos de refatoração para remover o test smell:	Separação dos casos de teste com @pytest.mark.parametrize("match_type", [...]): a lista local e o laço for foram removidos, e cada tipo de correspondência virou um caso de teste independente. A asserção é a mesma do teste original. O arquivo passa no pytest (25 passed, eram 21), e o TEMPY deixou de apontar o smell.

SMELL 33	
Colaborador:	Guilherme Sousa
Localização:	• Arquivo: backend/python/tests/unit/modules/agents/qna/test_schemas.py • Método: test_all_answer_match_types_are_defined • Linha: 195 • Commit: refactor(tests): remove Conditional Test Logic in test_all_answer_match_types_are_defined · fel0psmya/pipeshub-ai@c574aa6
Eu estou atualmente trabalhando na refatoração do seguinte test smell:	Conditional Test Logic. O método de teste monta uma lista local com os cinco tipos de correspondência e, dentro de um

	laço for, cria um dicionário AgentAnswerWithMetadataDict (TypedDict) para cada tipo e verifica o valor de answerMatchType. São cinco casos de teste em um único método, e uma falha em um tipo esconde o resultado dos tipos seguintes.
Minhas principais dificuldades na remoção do test smell são:	O teste tinha um comentário explicando o objetivo (“verificar que os cinco tipos são aceitos pela anotação do TypedDict”), que ficava dentro do corpo antes da lista. Com a lista indo para o decorator, foi preciso decidir onde manter esse comentário para que ele continuasse descrevendo o teste. Fora isso, é o mesmo padrão dos SMELLS 31 e 32, e mantive a mesma solução para o arquivo ficar consistente.
Usei o CATSPY para:	Confirmar que o laço for dentro do teste caracteriza Conditional Test Logic e aplicar a refatoração proposta, separando cada caso em um teste independente.
Página do CATSPY consultada:	CATSPY - Conditional Test Logic
Eu estou usando os seguintes métodos de refatoração para remover o test smell:	Separação dos casos de teste com @pytest.mark.parametrize("match_type", [...]): a lista local e o laço for foram removidos, cada tipo de correspondência virou um caso de teste independente, e o comentário que documenta o objetivo do teste foi mantido acima do decorator. O arquivo passa no pytest (29 passed, eram 25), e o TEMPY deixou de apontar o smell.

SMELL 34	
Colaborador:	Guilherme Sousa
Localização:	• Arquivo: backend/python/tests/unit/modules/agents/qna/test_schemas.py • Método: test_invalid_confidence_raises • Linha: 117 • Commit: refactor(tests): remove Unknown Test in test_invalid_confidence_raises · fel0psmya/pipeshub-ai@35aba66
Eu estou atualmente trabalhando na refatoração do seguinte test smell:	Unknown Test. O método de teste não tem nenhuma asserção explícita: ele só cria um AgentAnswerWithMetadataJSON com confidence="Ultra"

	dentro de pytest.raises(ValidationError). O teste confirma que algum erro de validação ocorreu, mas não verifica qual. Se o erro passasse a vir de outro campo, ele continuaria passando sem detectar a mudança.
Minhas principais dificuldades na remoção do test smell são:	Antes de escrever as asserções foi preciso descobrir o que exatamente o Pydantic devolve nesse caso. Executei a criação do modelo e inspecionei ValidationError.errors(), que retorna um único erro com loc == ("confidence",) e type == "literal_error". Outro cuidado: a primeira versão comparava os erros com uma compreensão de lista, e o TEMPY trata for dentro de teste como Conditional Test Logic. Para não introduzir um smell novo, usei asserções diretas.
Usei o CATSPY para:	Confirmar a definição de Unknown Test (teste sem asserções explícitas, que passa se nenhuma exceção for lançada) e seguir a refatoração proposta: introduzir asserções que validem explicitamente o resultado esperado.
Página do CATSPY consultada:	CATSPY - Unknown Test
Eu estou usando os seguintes métodos de refatoração para remover o test smell:	Adição de asserções explícitas. A exceção passou a ser capturada com pytest.raises(ValidationError) as exc_info, e o teste verifica que há exatamente um erro de validação, que ele está no campo confidence e que é do tipo literal_error (valor fora dos permitidos). O arquivo passa no pytest (29 passed), e o TEMPY deixou de apontar o smell.

SMELL 35	
Colaborador:	Guilherme Sousa
Localização:	• Arquivo: backend/python/tests/unit/modules/agents/qna/test_schemas.py • Método: test_invalid_answer_match_type_raises • Linha: 125 • Commit: refactor(tests): remove Unknown Test in test_invalid_answer_match_type_raises · fel0psmya/pipeshub-ai@cc23017

UNIVERSIDADE FEDERAL DO CEARÁ

Eu estou atualmente trabalhando na refatoração do seguinte test smell:	Unknown Test. O método de teste não tem nenhuma asserção explícita: ele só cria um AgentAnswerWithMetadataJSON com answerMatchType="Unknown Type" dentro de pytest.raises(ValidationError). Verifica que algum erro de validação ocorreu, mas não qual campo o causou nem por quê.
Minhas principais dificuldades na remoção do test smell são:	Garantir que as asserções verificassem o comportamento real do modelo, e não uma suposição. Inspecionei o ValidationError.errors() desse caso: um único erro, em loc == ("answerMatchType",), do tipo literal_error. Mantive o mesmo formato de asserções do SMELL 34, sem laços nem compreensões de lista, para não introduzir Conditional Test Logic e para manter o arquivo consistente.
Usei o CATSPY para:	Confirmar que a ausência de asserções caracteriza Unknown Test e aplicar a refatoração proposta: introduzir asserções que validem explicitamente o resultado esperado.
Página do CATSPY consultada:	CATSPY - Unknown Test
Eu estou usando os seguintes métodos de refatoração para remover o test smell:	Adição de asserções explícitas. A exceção passou a ser capturada com pytest.raises(ValidationError) as exc_info, e o teste verifica que há exatamente um erro de validação, no campo answerMatchType, do tipo literal_error. O arquivo passa no pytest (29 passed), e o TEMPY deixou de apontar o smell.

SMELL 36	
Colaborador:	Guilherme Sousa
Localização:	• Arquivo: backend/python/tests/unit/modules/agents/qna/test_schemas.py • Método: test_missing_required_answer_raises • Linha: 133 • Commit: refactor(tests): remove Unknown Test in test_missing_required_answer_raises · fel0psmya/pipeshub-ai@099e606
Eu estou atualmente trabalhando na refatoração do seguinte test smell:	Unknown Test. O método de teste não tem nenhuma asserção explícita: ele só cria um

UNIVERSIDADE FEDERAL DO CEARÁ

	AgentAnswerWithMetadataJSON sem o campo obrigatório answer, dentro de pytest.raises(ValidationError). O teste confirma que houve um erro de validação, mas não que o motivo foi a ausência de answer.
Minhas principais dificuldades na remoção do test smell são:	Diferente dos SMELLS 34 e 35, o erro esperado não é um valor inválido (literal_error), e sim um campo obrigatório ausente. Inspecionei o ValidationError.errors() desse caso para confirmar o tipo exato que o Pydantic usa: um único erro, loc == ("answer",) e type == "missing". Mantive o mesmo formato de asserções dos smells anteriores, sem laços, para não introduzir Conditional Test Logic.
Usei o CATSPY para:	Confirmar que a ausência de asserções caracteriza Unknown Test e aplicar a refatoração proposta: introduzir asserções que validem explicitamente o resultado esperado.
Página do CATSPY consultada:	CATSPY - Unknown Test
Eu estou usando os seguintes métodos de refatoração para remover o test smell:	Adição de asserções explícitas. A exceção passou a ser capturada com pytest.raises(ValidationError) as exc_info, e o teste verifica que há exatamente um erro de validação, no campo answer, do tipo missing (campo obrigatório ausente). O arquivo passa no pytest (29 passed), e o TEMPY deixou de apontar o smell.

SMELL 37	
Colaborador:	Guilherme Sousa
Localização:	• Arquivo: backend/python/tests/unit/modules/agents/qna/test_schemas.py • Método: test_missing_required_confidence_raises • Linha: 140 • Commit: refactor(tests): remove Unknown Test in test_missing_required_confidence_raises · fel0psmya/pipeshub-ai@b0009ef
Eu estou atualmente trabalhando na refatoração do seguinte test smell:	Unknown Test. O método de teste não tem nenhuma asserção explícita: ele só cria um AgentAnswerWithMetadataJSON sem o campo obrigatório confidence, dentro de pytest.raises(ValidationError). O teste confirma que houve um erro de validação, mas não que o

	motivo foi a ausência de confidence.
Minhas principais dificuldades na remoção do test smell são:	Este é o último dos quatro Unknown Test do arquivo, e a principal preocupação foi manter as asserções coerentes com as dos SMELLS 34 a 36. Também foi preciso confirmar, pela inspeção do ValidationError.errors(), que a ausência de confidence gera um único erro do tipo missing, e não erros em outros campos que dependem dele.
Usei o CATSPY para:	Confirmar que a ausência de asserções caracteriza Unknown Test e aplicar a refatoração proposta: introduzir asserções que validem explicitamente o resultado esperado.
Página do CATSPY consultada:	CATSPY - Unknown Test
Eu estou usando os seguintes métodos de refatoração para remover o test smell:	Adição de asserções explícitas. A exceção passou a ser capturada com pytest.raises(ValidationError) as exc_info, e o teste verifica que há exatamente um erro de validação, no campo confidence, do tipo missing (campo obrigatório ausente). O arquivo passa no pytest (29 passed), e o TEMPY não aponta mais nenhum smell em test_schemas.py.

SMELL 38	
Colaborador:	Guilherme Sousa
Localização:	• Arquivo: backend/python/tests/unit/modules/parsers/test_pptx_parser.py • Método: test_different_status_values_fail • Linha: 178 • Commit: refactor(tests): remove Unknown Test in test_different_status_values_fail · fel0psmya/pipeshub-ai@50b3cc5
Eu estou atualmente trabalhando na refatoração do seguinte test smell:	Unknown Test. O método de teste não tem nenhuma asserção explícita: ele só confirma, com pytest.raises(DocumentProcessingError), que parse_file lança uma exceção quando o status da conversão é diferente de “success”. Não verifica o conteúdo da exceção, ou seja, se a mensagem e os detalhes informam corretamente o motivo da falha.
Minhas principais dificuldades na	Este smell estava no mesmo método do SMELL 37

remoção do test smell são:	(Conditional Test Logic) e só pôde ser tratado depois dele, em um commit separado. Para escrever asserções corretas, li o código de produção de PPTXParser.parse_file, que lança a exceção com a mensagem "Failed to parse PPTX: <status>" e com details={"status": str(result.status)}. Como o status é um objeto mock, a asserção sobre details compara com str(mock_result.status), e não com um texto fixo. Também foi preciso posicionar as asserções fora do bloco pytest.raises, porque o código depois da linha que lança a exceção não é executado.
Usei o CATSPY para:	Confirmar que a ausência de asserções caracteriza Unknown Test e aplicar a refatoração proposta: introduzir asserções que validem explicitamente o resultado esperado.
Página do CATSPY consultada:	CATSPY - Unknown Test
Eu estou usando os seguintes métodos de refatoração para remover o test smell:	Adição de asserções explícitas. A exceção passou a ser capturada com pytest.raises(DocumentProcessingError) as exc_info, e o teste verifica que a mensagem começa com “Failed to parse PPTX” e que details traz o status retornado pelo conversor. O arquivo passa no pytest (15 passed), e o TEMPY deixou de apontar o smell.

SMELL 39	
Colaborador:	Guilherme Sousa
Localização:	• Arquivo: backend/python/tests/unit/modules/parsers/test_pptx_parser.py • Método: test_failure_raises_value_error (classe TestParseBinary) • Linha: 73 • Commit: refactor(tests): remove Unknown Test in test_failure_raises_value_error (parse_binary) · fel0psmya/pipeshub-ai@63d91fd
Eu estou atualmente trabalhando na refatoração do seguinte test smell:	Unknown Test. O método de teste não tem nenhuma asserção explícita. Ele só usa pytest.raises(DocumentProcessingError, match="Failed to

UNIVERSIDADE FEDERAL DO CEARÁ

	parse PPTX") para confirmar que parse_binary lança uma exceção quando a conversão falha. O match confere só o prefixo da mensagem e não verifica os detalhes da exceção.
Minhas principais dificuldades na remoção do test smell são:	Entender o que o código de produção coloca na exceção: a mensagem "Failed to parse PPTX: <status>" e details={"status": str(result.status)}. O teste sobrescreve o __str__ do status mock para devolver "failure", e por isso foi possível comparar a mensagem completa e o dicionário de detalhes com valores fixos. Também existe outro método com o mesmo nome na classe TestParseFile, então foi preciso deixar claro no commit qual dos dois foi refatorado.
Usei o CATSPY para:	Confirmar que a ausência de asserções caracteriza Unknown Test e aplicar a refatoração proposta: introduzir asserções que validem explicitamente o resultado esperado.
Página do CATSPY consultada:	CATSPY - Unknown Test
Eu estou usando os seguintes métodos de refatoração para remover o test smell:	Adição de asserções explícitas. A exceção passou a ser capturada com pytest.raises(DocumentProcessingError) as exc_info, e o teste verifica a mensagem completa (“Failed to parse PPTX: failure”) e o campo details ({“status”: “failure”}). O arquivo passa no pytest (15 passed), e o TEMPY deixou de apontar o smell.

SMELL 40	
Colaborador:	Guilherme Sousa
Localização:	• Arquivo: backend/python/tests/unit/modules/parsers/test_pptx_parser.py • Método: test_converter_exception_propagates • Linha: 86 • Commit: refactor(tests): remove Unknown Test in test_converter_exception_propagates · fel0psmya/pipeshub-ai@98a80d5
Eu estou atualmente trabalhando na refatoração do seguinte test smell:	Unknown Test. O método de teste não tem nenhuma asserção explícita. Ele só usa pytest.raises(RuntimeError,

	match="Corrupt file") para confirmar que uma exceção do conversor é lançada por parse_binary, sem verificar que ela é propagada sem alteração.
Minhas principais dificuldades na remoção do test smell são:	Definir qual asserção expressa o comportamento “propaga a exceção”. Comparar só o tipo e a mensagem não garante que o código de produção não tenha embrulhado ou recriado o erro. Para resolver isso, a exceção passou a ser criada antes em uma variável, e o teste compara a identidade do objeto capturado com ela.
Usei o CATSPY para:	Confirmar que a ausência de asserções caracteriza Unknown Test e aplicar a refatoração proposta: introduzir asserções que validem explicitamente o resultado esperado.
Página do CATSPY consultada:	CATSPY - Unknown Test
Eu estou usando os seguintes métodos de refatoração para remover o test smell:	Adição de asserções explícitas. A exceção passou a ser capturada com pytest.raises(RuntimeError) as exc_info. O teste verifica que exc_info.value is converter_error, ou seja, que é o mesmo objeto lançado pelo conversor, e que o conversor foi chamado uma vez (assert_called_once). O arquivo passa no pytest (15 passed), e o TEMPY deixou de apontar o smell.

SMELL 41	
Colaborador:	Guilherme Sousa
Localização:	• Arquivo: backend/python/tests/unit/modules/parsers/test_pptx_parser.py • Método: test_failure_raises_value_error (classe TestParseFile) • Linha: 131 • Commit: refactor(tests): remove Unknown Test in test_failure_raises_value_error (parse_file) · fel0psmya/pipeshub-ai@b9f3ba2
Eu estou atualmente trabalhando na refatoração do seguinte test smell:	Unknown Test. O método de teste não tem nenhuma asserção explícita. Ele só usa

	pytest.raises(DocumentProcessingError, match="Failed to parse PPTX") para confirmar que parse_file lança uma exceção quando a conversão falha. O match confere só o prefixo da mensagem e não verifica os detalhes da exceção.
Minhas principais dificuldades na remoção do test smell são:	Este método tem o mesmo nome do teste refatorado no SMELL 39 (classe TestParseBinary), então foi preciso identificar a classe correta e deixar isso explícito na mensagem do commit. Também foi preciso ver que, neste teste, o __str__ do status mock devolve "error", e não "failure", para escrever os valores esperados certos na mensagem e no details.
Usei o CATSPY para:	Confirmar que a ausência de asserções caracteriza Unknown Test e aplicar a refatoração proposta: introduzir asserções que validem explicitamente o resultado esperado.
Página do CATSPY consultada:	CATSPY - Unknown Test
Eu estou usando os seguintes métodos de refatoração para remover o test smell:	Adição de asserções explícitas. A exceção passou a ser capturada com pytest.raises(DocumentProcessingError) as exc_info, e o teste verifica a mensagem completa (“Failed to parse PPTX: error”) e o campo details ({“status”: “error”}). O arquivo passa no pytest (15 passed), e o TEMPY deixou de apontar o smell.

SMELL 42	
Colaborador:	Guilherme Sousa
Localização:	• Arquivo: backend/python/tests/unit/modules/parsers/test_pptx_parser.py • Método: test_file_not_found_propagates • Linha: 143 • Commit: refactor(tests): remove Unknown Test in test_file_not_found_propagates · fel0psmya/pipeshub-ai@b563d0b
Eu estou atualmente trabalhando na refatoração do seguinte test smell:	Unknown Test. O método de teste não tem nenhuma asserção explícita. Ele só usa

	pytest.raises(FileNotFoundError, match="No such file") para confirmar que parse_file deixa passar o erro de arquivo inexistente, sem verificar que a exceção é propagada sem alteração nem com qual caminho o conversor foi chamado.
Minhas principais dificuldades na remoção do test smell são:	Escolher asserções que realmente expressem o comportamento “propaga a exceção”, e não só repetir o tipo do erro. Para isso, o erro do conversor passou a ser criado em uma variável, e o teste compara a identidade do objeto capturado. Também mantive o mesmo padrão do SMELL 40, para o arquivo ficar consistente entre parse_binary e parse_file.
Usei o CATSPY para:	Confirmar que a ausência de asserções caracteriza Unknown Test e aplicar a refatoração proposta: introduzir asserções que validem explicitamente o resultado esperado.
Página do CATSPY consultada:	CATSPY - Unknown Test
Eu estou usando os seguintes métodos de refatoração para remover o test smell:	Adição de asserções explícitas. A exceção passou a ser capturada com pytest.raises(FileNotFoundError) as exc_info. O teste verifica que exc_info.value is converter_error, ou seja, que é o mesmo objeto lançado pelo conversor, e que o conversor foi chamado com o caminho do arquivo (assert_called_once_with("/nonexistent/file.pptx")). O arquivo passa no pytest (15 passed), e o TEMPY deixou de apontar o smell (0 smells no arquivo).

SMELL 43	
Colaborador:	Guilherme Sousa
Localização:	• Arquivo: backend/python/tests/unit/modules/parsers/test_pdf_processors_extended.py • Método:

UNIVERSIDADE FEDERAL DO CEARÁ

	test_detect_text_regions_filters_small_and_in_table • Linha: 328 • Commit: refactor(tests): remove Non-Functional Statement in test_detect_text_regions_filters_small_and_in_table · fel0psmya/pipeshub-ai@04ebb62
Eu estou atualmente trabalhando na refatoração do seguinte test smell:	Non-Functional Statement. Na linha 328 havia o comentário "# Only the large one should pass", que não tem efeito na execução. Ele descreve um comportamento esperado (só a região de texto grande deve ser mantida), mas o teste não verificava isso: a única asserção era isinstance(result, list).
Minhas principais dificuldades na remoção do test smell são:	Entender por que o TEMPY apontou esta linha, já que o teste não tem nenhum pass. Analisando o detector do TEMPY, vi que ele marca qualquer linha terminada em "pass", e por isso o comentário foi identificado. Outra dificuldade é que a classe TestOpenCVLayoutAnalyzerAnalyzePage está marcada com @pytest.mark.skip, porque OpenCVLayoutAnalyzer não existe mais no código de produção. Assim, o teste não é executado, e a nova asserção não pôde ser validada em execução.
Usei o CATSPY para:	Entender que trechos sem efeito no teste caracterizam Non-Functional Statement e aplicar a refatoração proposta: em vez de só apagar o trecho, transformar a intenção dele em uma asserção.
Página do CATSPY consultada:	CATSPY - Non-Functional Statement
Eu estou usando os seguintes métodos de refatoração para remover o test smell:	Substituição do statement não funcional por uma asserção explícita. O comentário foi removido, e a expectativa que ele descrevia passou a ser verificada com assert len(result) == 1. O arquivo continua com o mesmo resultado no pytest (10 passed, 27 skipped), e o TEMPY deixou de apontar o smell.

SMELL 44	
Colaborador:	Guilherme Sousa

UNIVERSIDADE FEDERAL DO CEARÁ

Localização:	• Arquivo: backend/python/tests/unit/modules/parsers/test_pdf_processors_extended.py • Método: test_analyze_page_text_region_classified_as_text • Linha: 46–91 • Commit: refactor(tests): remove Verbose Test in test_analyze_page_text_region_classified_as_text · fel0psmya/pipeshub-ai@31ef8b0
Eu estou atualmente trabalhando na refatoração do seguinte test smell:	Verbose Test. O método de teste tinha 46 linhas, e a maior parte delas era preparação. A página PDF falsa era montada com tamanho, imagem renderizada e lista de imagens, e o OpenCV falso era configurado para não detectar contornos. Isso escondia o que o teste realmente verifica: que um bloco de texto comum é classificado como TEXT.
Minhas principais dificuldades na remoção do test smell são:	Identificar quais partes da preparação eram genéricas e quais eram específicas do cenário testado. Só o conteúdo do get_text (o bloco de texto) e as asserções são próprios deste teste; o resto se repete em outros testes da classe. Outra dificuldade é que a classe está marcada com @pytest.mark.skip, porque OpenCVLayoutAnalyzer não existe mais no código de produção, então a validação ficou limitada à coleta do pytest e à análise do TEMPY.
Usei o CATSPY para:	Entender o critério do Verbose Test (método de teste longo, com várias responsabilidades) e aplicar a refatoração proposta: separar o teste em partes menores e mais focadas e levar a preparação comum para um ponto reutilizável, como o catálogo faz com o setUp.
Página do CATSPY consultada:	CATSPY - Verbose Test
Eu estou usando os seguintes métodos de refatoração para remover o test smell:	Extração da preparação para métodos auxiliares. A montagem da página falsa foi para _make_blank_page(), e a configuração do OpenCV sem contornos foi para _stub_cv2_without_contours(mock_cv2). Assim, o teste mantém só o cenário específico e as asserções. Ele caiu de 46 para 22 linhas, o pytest mantém o mesmo resultado (10 passed, 27 skipped), e o TEMPY deixou de apontar o smell.

UNIVERSIDADE FEDERAL DO CEARÁ

SMELL 45	
Colaborador:	Guilherme Sousa
Localização:	• Arquivo: backend/python/tests/unit/modules/parsers/test_pdf_processors_extended.py • Método: test_analyze_page_unclaimed_block_as_heading • Linha: 94–140 • Commit: refactor(tests): remove Verbose Test in test_analyze_page_unclaimed_block_as_heading · fel0psmya/pipeshub-ai@5eabc5f
Eu estou atualmente trabalhando na refatoração do seguinte test smell:	Verbose Test. O método de teste tinha 47 linhas, e boa parte delas repetia a preparação da página PDF falsa e do OpenCV falso. Isso deixava em segundo plano o que o teste realmente verifica: que um bloco de texto com fonte grande, não reivindicado pelo OpenCV, é classificado como HEADING.
Minhas principais dificuldades na remoção do test smell são:	Confirmar que a preparação deste teste era equivalente à que já tinha sido extraída no SMELL 44, para poder reaproveitar os mesmos métodos auxiliares sem mudar o cenário testado. Também foi preciso manter no teste os comentários sobre o cálculo do limite de fonte, que explicam por que esses valores de tamanho foram escolhidos. Assim como no SMELL 44, a classe está marcada com @pytest.mark.skip, então a validação ficou limitada à coleta do pytest e à análise do TEMPY.
Usei o CATSPY para:	Aplicar a refatoração proposta para Verbose Test: deixar o teste menor e focado em uma responsabilidade, levando a preparação comum para um ponto reutilizável.
Página do CATSPY consultada:	CATSPY - Verbose Test
Eu estou usando os seguintes métodos de refatoração para remover o test smell:	Reutilização dos métodos auxiliares de preparação _make_blank_page() e _stub_cv2_without_contours(mock_cv2), extraídos no SMELL 44, no lugar das linhas repetidas. O teste caiu de 47 para 31 linhas, o pytest mantém o mesmo resultado (10 passed, 27 skipped), e o TEMPY deixou de apontar o smell.

UNIVERSIDADE FEDERAL DO CEARÁ

SMELL 46	
Colaborador:	Guilherme Sousa
Localização:	• Arquivo: backend/python/tests/unit/modules/transformers/test_block_container_validator.py • Método: _assert_raises_with_codes • Linha: 159 • Commit: refactor(tests): remove Undefined Test in _assert_raises_with_codes · fel0psmya/pipeshub-ai@eacb10e
Eu estou atualmente trabalhando na refatoração do seguinte test smell:	Undefined Test. A função _assert_raises_with_codes contém asserções (pytest.raises e assert), mas o nome não segue nenhuma convenção que o framework e as ferramentas reconheçam: não começa com test nem é identificada como uma asserção customizada. Por isso ela era apontada como um teste não reconhecido.
Minhas principais dificuldades na remoção do test smell são:	Perceber que a função não é um teste perdido, e sim uma função auxiliar de asserção usada por 42 testes. Renomeá-la para test_... faria o pytest coletá-la como teste e falhar, e removê-la quebraria os testes que dependem dela. Analisando o detector do TEMPY, vi que métodos que começam com assert são tratados como asserções customizadas. Outra dificuldade é que a troca de nome exigiu atualizar as 42 chamadas e, como efeito colateral, também eliminou 41 ocorrências de Unknown Test no mesmo arquivo, que existiam só porque as chamadas a _assert_raises_with_codes não eram reconhecidas como asserção.
Usei o CATSPY para:	Entender que o Undefined Test acontece quando o nome de um método com asserções não segue a convenção do framework, e aplicar a refatoração proposta pelo catálogo: renomear o método.
Página do CATSPY consultada:	CATSPY - Undefined Test
Eu estou usando os seguintes métodos de refatoração para remover o test smell:	Renomeação do método para a convenção de asserção customizada: _assert_raises_with_codes virou assert_raises_with_codes, com a atualização de todas as chamadas. O comportamento não mudou, o pytest continua com 135 testes passando (nenhum teste novo coletado), e o TEMPY deixou de apontar o smell.

UNIVERSIDADE FEDERAL DO CEARÁ

SMELL 47	
Colaborador:	Guilherme Sousa
Localização:	• Arquivo: backend/python/tests/unit/modules/transformers/test_block_container_validator.py • Método: test_children_block_range_huge_end_single_error • Linha: 347 • Commit: refactor(tests): remove Conditional Test Logic in test_children_block_range_huge_end_single_error · fel0psmya/pipeshub-ai@ef2640b
Eu estou atualmente trabalhando na refatoração do seguinte test smell:	Conditional Test Logic. O teste usava uma list comprehension com for e if para filtrar os erros pelo código CHILDREN_BLOCK_INDEX_OUT_OF_RANGE antes das asserções. Essa lógica condicional dentro do teste o deixa menos direto e pode esconder erros extras, que seriam descartados pelo filtro.
Minhas principais dificuldades na remoção do test smell são:	Saber se dava para tirar o filtro sem mudar o que o teste verifica. Para isso, executei o validador com o mesmo cenário e confirmei que ele devolve exatamente um erro, com o código e a mensagem esperados. Assim, as asserções puderam ser feitas direto sobre a lista completa, sem filtro. O resultado é um teste mais rigoroso, que corresponde ao nome do teste (“single error”).
Usei o CATSPY para:	Entender que estruturas condicionais e de repetição dentro do teste caracterizam Conditional Test Logic e aplicar a refatoração proposta: eliminar a lógica condicional e verificar o resultado de forma direta.
Página do CATSPY consultada:	CATSPY - Conditional Test Logic
Eu estou usando os seguintes métodos de refatoração para remover o test smell:	Remoção da lógica condicional com asserções diretas. O filtro foi eliminado, e o teste passou a verificar a lista completa: exatamente um erro (len(errors) == 1), com o código CHILDREN_BLOCK_INDEX_OUT_OF_RANGE

	e com a mensagem contendo o intervalo [0, 1000000000]. O pytest continua com 135 testes passando, e o TEMPY deixou de apontar o smell.

SMELL 48	
Colaborador:	Guilherme Sousa
Localização:	• Arquivo: backend/python/tests/unit/modules/transformers/test_block_container_validator.py • Método: test_children_group_range_huge_end_single_error • Linha: 372 • Commit: refactor(tests): remove Conditional Test Logic in test_children_group_range_huge_end_single_error · fel0psmya/pipeshub-ai@db093d8
Eu estou atualmente trabalhando na refatoração do seguinte test smell:	Conditional Test Logic. O teste usava uma list comprehension com for e if para filtrar os erros pelo código CHILDREN_GROUP_INDEX_OUT_OF_RANGE antes de verificar a quantidade. Essa lógica condicional deixa o teste menos direto e pode esconder erros extras, que o filtro descartaria.
Minhas principais dificuldades na remoção do test smell são:	Garantir que tirar o filtro não mudaria o que o teste verifica. Rodei o validador com o mesmo cenário e confirmei que ele devolve um único erro, com o código e o intervalo esperados. Também vi que o teste original só conferia a quantidade de erros. Por isso acrescentei as verificações de código e de mensagem, para ficar igual ao teste equivalente de blocos (SMELL 47).
Usei o CATSPY para:	Entender que estruturas condicionais e de repetição dentro do teste caracterizam Conditional Test Logic e aplicar a refatoração proposta: eliminar a lógica condicional e verificar o resultado de forma direta.
Página do CATSPY consultada:	CATSPY - Conditional Test Logic
Eu estou usando os seguintes métodos de refatoração para remover o test smell:	Remoção da lógica condicional e uso de asserções diretas. Tirei o filtro, e agora o teste confere a lista completa: exatamente um erro, com o código

	CHILDREN_GROUP_INDEX_OUT_OF_RANGE e com a mensagem contendo o intervalo [0, 1000000000]. O pytest continua com 135 testes passando, e o TEMPY deixou de apontar o smell.

SMELL 49	
Colaborador:	Guilherme Sousa
Localização:	• Arquivo: backend/python/tests/unit/modules/parsers/test_pptx_parser.py • Método: test_different_status_values_fail • Linha: — • Commit: refactor(tests): remove Conditional Test Logic in test_different_stat… · fel0psmya/pipeshub-ai@d870680
Eu estou atualmente trabalhando na refatoração do seguinte test smell:	Conditional Test Logic. O método de teste usa um laço for para percorrer quatro status de conversão diferentes de “success” (“partial”, “error”, “timeout”, “unknown”). A cada volta, monta os mocks do DocumentConverter e verifica que parse_file lança DocumentProcessingError. São quatro casos de teste dentro de um único método: se um status falhar, os seguintes não são executados e o relatório não mostra qual status causou a falha.
Minhas principais dificuldades na remoção do test smell são:	O mesmo método também tem um segundo smell, Unknown Test, porque não há asserção explícita. Pela regra de 1 commit por smell, este commit trata apenas do laço, e o Unknown Test fica para o commit seguinte. Também foi preciso garantir que os mocks continuassem sendo criados do zero para cada status, o que acontecia dentro do laço e agora acontece em cada caso parametrizado, para que um caso não interfira no outro.
Usei o CATSPY para:	Confirmar que o laço for dentro do teste caracteriza Conditional Test Logic e aplicar a refatoração proposta: separar cada caso em um teste independente.
Página do CATSPY consultada:	CATSPY - Conditional Test Logic
Eu estou usando os seguintes métodos de refatoração para	Separação dos casos de teste com @pytest.mark.parametrize("status_val", [...]): o laço for foi

remover o test smell:	removido e cada status virou um caso de teste independente, com seus próprios mocks. O arquivo passa no pytest (15 passed, eram 12, porque o teste virou 4 casos), e o TEMPY deixou de apontar o Conditional Test Logic.